Chapter 3: Data Types, Variables, Literals, Type Casting, Assignments, Operator

- Data Types
- Variables
- Literals
- Type Casting
- Assignments
- Operators

Data Types

- Data Types in a programming language specifies the type of data to be stored and the size of memory to be allocated.
- Data Types can be classified as:
  1. Fundamental Data Types
  2. Reference Types

Fundamental Data Types

- Fundamental data types are the data types that the compiler understands.
- These are also referred as primitive, pre-defined or built in data types.

Data Types	Size in Bytes	Size in Bits	Range
char	2	16	0 to 65,535
byte	1	8	–128 to 127
short	2	16	–32,768 to 32,767
int	4	32	–2,147,483,648 to 2,147,483,647
long	8	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4	32	4.9e–324 to 1.8e+308
double	8	64	1.4e–045 to 3.4e+038
boolean	virtual machine dependent		true, false

Variables

- Variables are temporary memory locations used to store a value.
- The value stored in a variable can change/vary.
- Variable naming rules:
  - It should start with an alphabet, underscore (_) or dollar ($)
  - It can contain alphabets, digits, underscore (_) or dollar ($)
  - No spaces, no special characters except underscore (_) and dollar ($)
  - It should not be a keyword
  - It should be unique within its scope
- Variable naming conventions:
  - Variables in Java make use of camelCase.
    Example: myVar, oneTwo, oneTwoThree

Variable Declaration

```
char c;
int num;
```

Variable Initialization

```
c = 'a';
num = 10;
```

Variable Declaration and Initialization

```
char c = 'a';
int num = 10;
```

Literals

Literals are used to specify data type of a value.
Literals are not used with variables.
There are no literals for byte and short.
Literals are case insensitive.

Literals values for all primitive data types

```
24                      //int
124.543                 // double
true                    // boolean
'v'                     //char
```

Integer Literals

An integer literal can be represented in the following forms:

- Decimal (base 10)
- Octal (base 8)
- Hexadecimal (base 16)

Decimal Literals

```
int num = 21, 99_123;
```

There is no prefix or suffix

Octal Literals

- Numbers can be from 0 to 7
- Prefix used is 0

```
int num = 05;
```

Hexadecimal Literals

- Numbers can be from 0 to 9, a to f
- Prefix used is 0x

NOTE: a to f is not case sensitive, a is equal t o A

```
int a = 0x5;
int b = 0xF;
long a = 05l;           // long
long b = 0xFl;          // long
```

ascii → total 256
unicode जैसा single quote में लिखते हैं।
↳ range as char 0 to 65535

Floating Point Literals

```
float pi = 3.14;                    // Incorrect
float pi = 3.14f;
double pi = 3.14;
double pi = 3.14d;
```

Boolean Literals

```
boolean married = true;             // Correct
boolean married = 1;                // Incorrect

if(1) {                             // Incorrect
        // statements
}
```

Character Literals → it can also represented as integral literal.

```
char c = 'A';                       char a = 106;
char value = 'y';
char newline = '\n';
```

↳ इसके बाद 4 ~~decimal~~ digit के hexadecimal number.

NOTE: Character literals can also be written in unicode

```
char c = '\u004A';
```

↳ ये बता रहा है कि unicode है।

String Literals

```
String str = "Hello World";
String str = "Hell\u0001 W\u0001rld";
```

TypeCasting

TypeCasting is converting from one data type to another data type

It can be inform of:

1. **Narrowing/Explicit**: Assigning larger data types to smaller data types
2. **Widening/Implicit**: Assigning smaller data types to larger data types

```
int i = 10;
long j = 20;
i = j;                              // Incorrect
i = (int) j;                        // Correct
j = i;                              // Correct
j = (long) i;                       // Correct
```

Assignments

Rule 1: Precedence (double, float, long, int)

```
class Test {
        public static void main(String args[]) {
                byte b1 = 10;
```

valid literal.
char a = '\u0100'
char b = 0Xffaa
char c = '\uabcd'
char d = 65536X → not in range

integer

```
        byte b2 = 20;
        byte b3 = b1 + b2;
        System.out.println(b3);
    }
}
```

Rule 2: Range Rule (Applicable on byte, short and char)

```
class Test {
    public static void main(String args[]) {
        char b = 65536;
        System.out.println(b);
    }
}
```

a+ = 1 by → int
a = a+1
2++
2 = 2+1

Rule 3: Shortcut Assignment Operators (+=, -=)

- Range rules does not applies
- if outside range, it becomes cyclic

```
class Test {
    public static void main(String args[]) {
        byte b = 122;
        b += 6;
        System.out.println(b);   //-1 28
    }
}
```

Rule 4: Variables are evaluated at runtime, whereas values are evaluated at compiletime

Error:

```
class Test {
    public static void main(String args[]) {
        byte b, b1 = 10;
        b = b1 + b1;
        System.out.println(b);
    }
}
```

Will Compile:

```
class Test {
    public static void main(String args[]) {
        byte b;
        b = 10 + 10;
        System.out.println(b);
    }
}
```

Operators

An operator is a symbol that allows a programmer to perform certain arithmetic or logical operations on data and variables.

1. Arithmetic (*,/,%,+,-,++,--)
2. Assignment (= , +=, -=, *=, /= , %=)
3. Relations (==, !=, <, > , <= , >=)
4. Instance of (instanceof)
5. Logical (&, |, ^, !, &&, | |)
6. Conditional (? :)

Example:

```
class Op1 {
        public static void main(String args[]) {
                int a = 5;
                int b = 3;
                int c = 0;
                c = a + b;
                System.out.println(c);
                c = a - b;
                System.out.println(c);
                c = a * b;
                System.out.println(c);
                c = a / b;
                System.out.println(c);
                c = a % b;
                System.out.println(c);
        }
}
```

Example:

```
class Op2 {
        public static void main(String args[]) {
                int a = 6, b = 5;
                a = b++;
                System.out.println(a);
                System.out.println(b);
                a = ++b;
                System.out.println(a);
                System.out.println(b);
                b++;
                System.out.println(b);
                System.out.println(b++);
        }
}
```

Example:

```
class Op3 {
        public static void main(String args[]) {
                int a = 5;
                int b = 6;
                if ((b == a) && (a < ++b))
                        System.out.println("Here");
                System.out.println(" a : " + a + " b : " + b);
        }
}
```

Example:

```
class Op4 {
        public static void main(String args[]) {
                String str = " ";
                int a = 333;
                int b = 666;
                System.out.println(str + a + b);
                System.out.println(str + (a + b));
                System.out.println(a + b + str);
                str += 10;
                System.out.println(str);
        }
}
```

Example:

```
class Op5 {
        public static void main(String args[]) {
                int a = 10;
                a = a + 5 * 15;
                System.out.println(a);
        }
}
```

Example:

```
class Op6 {
        public static void main(String args[]) {
                String str = "Welcome";
                str += " to ";
                str += "Java";
                System.out.println(str);
        }
}
```

Example:

```
class Op7 {
        public static void main(String args[]) {
                int a = 10;
                int b = 20;
                int c = 30;
                int d = a > b & a > c ? a : b > c ? b : c;
                System.out.println(d);
        }
}
```

Example:

```
class Op8 {
        public static void main(String args[]) {
                int a = 9;
                float b = 9.0f;
                double c = 9.0;
                if (a == b && a == c)
                        System.out.println("Equal");
                else
                        System.out.println("Not Equal");
        }
}
```

Example:

```
class Op9 {
        public static void main(String args[]) {
                int a = 65;
                char b = 'A';
                System.out.println(b);
                if (a == b)
                        System.out.println("Equal");
                else
                        System.out.println("Not Equal");
        }
}
```

Assignment

Theory Questions

1. How are data types classified? Explain the fundamental data types that Java provides.
2. What are variables? Explain variable naming rules and conventions.
3. What is type casting? Explain its types.
4. What are literals? Explain the types of literals that Java provides.
5. List the various operators that Java supports.

Chapter 4: Input and Output, Conditional and Looping Structures

- Scanner Class
- Conditional Structures – if and switch
- Looping Structures - while, do while, for, enhanced for
- Jumping Statements

Scanner Class

- To take input from user "Scanner" class is used
- This "Scanner" class comes from java.util package
- To use "Scanner" class in Java program, statement is
  - import java.util.Scanner;
- To associate Scanner to keyboard (create an object)
  Scanner a = new Scanner(System.in);

To accept a word

```
String word = a.next();
```

To accept a line

```
String line = a.nextLine();
```

To accept a character

```
char c = a.nextLine().charAt(0);        or        char c = a.next().charAt(0);
```

To accept an integer

```
int num = a.nextInt();
```

To accept a float

```
float fp = a.nextFloat();
```

To accept a double

```
double d = a.nextDouble();
```

Write a program to accept your name and address and display it on Screen

```
import java.util.Scanner;
class Test1 {
        public static void main(String args[]) {
                String name, add;
                Scanner a = new Scanner(System.in);
                System.out.println("Enter Name :");
                name  = a.nextLine();
                System.out.println("Enter Address :");
                add  = a.nextLine();
                System.out.println("Name is : "  + name);
                System.out.println("Address is : " + add);
```

Test t1 = new Test();

```
    }
}
```

Write a program to radius of a circle and print its area and circumference

```
import java.util.Scanner;
class Test2 {
        public static void main(String args[]) {
                float radius, area = 0,cir = 0;
                Scanner a = new Scanner(System.in);
                System.out.println("Enter Radius :");
                radius  = a.nextFloat();
                area = 3.14f * radius * radius;
                // area = (float) (Math.PI * Math.pow(radius, 2));
                cir = 2 * 3.14f * radius;
                System.out.println("Area is : " + area);
                System.out.println("Circumference is : " + cir);
        }
}
```

Conditional / Decision / Selection Structures

- To select single out of multiple options
- Like C/C++ even Java provides:
    1. if
    2. switch
- if cane be used in 3 ways

valid expression: byte, short, int, char, String

Invalid ex: boolean, long, float, double

```
if(<cond>) {
//T-Part
}
```

```
if(<cond1>) {
//T-Part
}
else {
//F-Part
}
```

```
if(<cond1>) {
//T-Part of cond1
}
else if (<cond2>) {
//T-Part of con2
}
:
else {
//F-Part
}
```

```
switch(<variable>) {
        case <value1> : // T-Part of Value 1
                        break;
        case <value2> : // T-Part of Value 2
                        break;
        case <value3> : // T-Part of Value 3
                        break;
        :
        default :       // F-Part
}
```

break, default are optional in switch

Write a program to accept a character and print whether it is upper case, lower case or a special character

```
import java.util.Scanner;
class Test3 {
	public static void main(String args[]) {
		char c;
		Scanner a = new Scanner(System.in);
		System.out.println("Enter Character :");
		c = a.nextLine().charAt(0);
		if(c >= 65 && c <= 90)
			System.out.println("Upper Case");
		else if(c >= 'a' && c <= 'z')
			System.out.println("Lower Case");
		else
			System.out.println("Special");
	}
}
```

Write a program to accept a character and print whether it is a vowel or a consonant

```
import java.util.Scanner;
class Test4 {
	public static void main(String args[]) {
		char c;
		Scanner a = new Scanner(System.in);
		System.out.println("Enter Character :");
		c = a.nextLine().charAt(0);
		switch(c) {
			case 'a' : System.out.println("Vowel a.");
				break;
			case 'e' : System.out.println("Vowel e.");
				break;
			case 'i' : System.out.println("Vowel i.");
				break;
			case 'o' : System.out.println("Vowel o.");
				break;
			case 'u' : System.out.println("Vowel u.");
				break;
			default : System.out.println("It is a consonant.");
		}
	}
}
```

हर binary operator के आगे-पीछे space होना चाहिए
switch के अन्दर variable का नाम आता है.

Looping/Repetitive Structures

It is used to repeat set of statements or block of code

Like C/C++, even Java provides:

1. while
2. do ... while
3. for

```
//initialization
while(<cond>) {
//statements
//re-initialization
}
```

```
//initialization
do {
//statements
//re-initialization
} while(<cond>);
```

```
for(<initialization>;<condition>;<re-init>) {
//statements
}
```

Parts of looping structure:

1. Initialization : Defines the starting value
2. Condition: Specifies the ending value
3. Re-Initialization : Defines the step value

```
class Test4 {
        public static void main(String args[]) {
                int i;
                i = 1;
                while(i <= 100) {
                        System.out.print("\t" + i) ;
                        i++;
                }
                do {
                        System.out.print("\t" + i);
                        i++;
                } while( i<= 200);
                for(;i = 300;i++)
                        System.out.print("\t" + i);
        }
}
```

Enhanced For OR for each

- it was introduced with jdk5
- it is used to enumerate arrays and collections

```
class Test {
        public static void main(String args[]) {
                int num[] = {10, 20, 30, 40, 50};
                for(int x : num)
                        System.out.println(x);
                String weekDays[] = { "Monday" ,  "Tuesday", "Wednesday", "Thursday", "Friday"};
                for(String w : weekDays)
                        System.out.println(w);
```

```
    }
}
```

Jumping Statements

- break
- continue – immediate loop will continue. (वापस cursor को लेके चले जाता है)
- return

Example:

```
class Jump1 {
        public static void main(String args[]) {
        int j = 0;
        while(true) {
                j = j + 2;
                if(j == 6) {
                        break;
                }
                System.out.println(j);
                }
        }
}
```

Example:

```
class Jump2 {
        public static void main(String args[]) {
                int j, k;
                abc:
                for(j = 0 ; j < 5 ; j++) {
                        System.out.println("j= " + j);
                        for( k = 2 ; k < 5 ; k++) {
                                System.out.println("k= " + k);
                                if( j == k) {
                                        break  abc;
                                }
                        }
                }
        }
}       // abc: (label) should be immediate before the loop
```

→ ये भी दी से ज्यादा loop से बाहर जाना है.

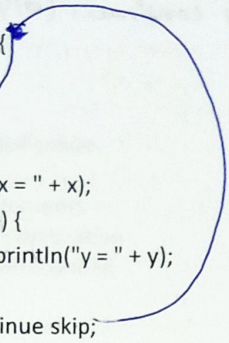
Example:

```
class Jump3 {
        public static void main(String args[]) {
                for(int i = 2; i <= 10 ; i+=2) {
                        if( i == 6) {
                                continue;
                        }
                        System.out.println(i);
                }
```

```
    }
}
```

Example:

```
class Jump4 {
    public static void main(String args[]) {
        int x , y;
        skip:
        for( x = 2 ; x  < 4 ; x++) {
            System.out.println("x = " + x);
            for( y = 2 ; y < 6 ; y++) {
                System.out.println("y = " + y);
                if(x == y) {
                    continue skip;
                }
            }
        }
    }
}
```



Example:

```
class Jump5 {
    public static void main(String args[]) {
        int input = 10;
        int result = input * input;
        System.out.println(result);
        return;
    }
}
```

Assignment

Theory Questions

1. What is the software requirement for Java? Name some of the IDEs used for Java development.
2. Why to be set the PATH variable? How do we set PATH for Java?
3. What is the different between byte code and machine code?

Practical Questions

1. Write a program that displays **Hello World** on screen.
2. Write a program that displays your name on screen within double quotes.
3. Write a program that accepts your name, address and mobile number and display it on screen.
4. Write a program that accepts radius of a circle and display its area and circumference.
5. Write a program that accepts principle, rate and time and display simple and compound interest.
6. Write a program that accepts width and height of a rectangle and display its area and perimeter.
7. Write a program that accepts temperature in Centigrade and display it in Fahrenheit. (F=C*9/5+32)

किसी class को private और protected नहीं कर सकते हैं

Chapter 5: Object Oriented Programming Structure (OOPS)

- Classes and Objects
- Constructors
- Garbage Collection
- Inheritance
- static
- final
- abstract

Classes and Objects

Class: It is used to create a user define/real time data type.

- It is a collection of member data/attributes and member functions/methods.
- Member data specifies the type of data to be stored.
- Member functions acts as a mediator between user and data.
- Class implements Encapsulation and abstraction.
- Set of rules/specification/layout for user define data type

Object: Instances of class

- Store data and invoke methods.
- Implementation of Rules

Syntax: Class

```
<AccessSpecifier> class <ClassName> {
        //Member data
        // Member functions
}
```

Access Specifier:

- There are 4 type access specifiers in Java.
- These access specifier specifies the scope of member data, member functions, classes, interface and enums

1. private: Within class
2. protected: Within class and derive classes
3. package: Within directory/folder (package is the default access specifier)
4. public: Anywhere

NOTE: All predefined classes make use of TitleCase

Syntax: Object

```
<ClassName> <objectName> = new <ClassName>();          //Declaration and initialization
OR
<ClassName> <objectName>;                              //Declaration
<objectName> = new <ClassName>();                      //Initialization
```

NOTE: Objects makes use of camelNotation

in Javagroup access specifier is not available.
with get/set we have options for read or write or both. But public data always have both the options (read as well as write)
② It is possible to validate using set. But, public data cannot be validate the data.

Example: Point

```
import java.util.Scanner;
class Point {
        private int x;
        private int y;
        public int getX () {
                return x;
        }
        public void setX (int a) {
                x = a;
        }
        public int getY () {
                return y;
        }
        public void setY (int a) {
                y = a;
        }
        public void acceptData() {
                Scanner a = new Scanner(System.in);
                System.out.println("Enter X  : ");
                x = a.nextInt();
                System.out.println("Enter Y  : ");
                y = a.nextInt();
        }
        public void showData() {
                System.out.println("X : " + x);
                System.out.println("Y : " + y);
        }
        public static void main(String args[]) {
                Point p = new Point();
                p.acceptData();
                p.showData();
                Point p1;
                p1 = new Point();
                p1.setX (10);
                p1.setY (20);
                p1.showData();
        }
}
```

Example: Rectangle

```
import java.util.Scanner;
class Rectangle {
```

```
    private int width;
    private int height;
    public int getWidth() {
            return width;
    }
    public void setWidth(int w) {
            width = w;
    }
    public int getHeight() {
            return height;
    }
    public void setHeight(int h) {
            height = h;
    }
    public void acceptData() {
            Scanner a = new Scanner(System.in);
            System.out.println("Enter Width");
            width = a.nextInt();
            System.out.println("Enter Height");
            height = a.nextInt();
    }
    public void showData() {
            System.out.println("Width :" + width);
            System.out.println("Height :" + width);
    }
    public int getArea() {
            return width * height;
    }
    public int getPerimeter() {
            return 2 * (width + height);
    }
    public static void main(String args[]) {
            Rectangle r = new Rectangle();
            r.acceptData();
            r.showData();
            System.out.println("Area is : " + r.getArea());
    }
}
```

Constructors

- Constructors are used to initialization of User define data types in various forms.
- Constructors are special member functions:
  - Same name as of a class.
  - Constructors are executed automatically/implicitly whenever an object is created (new)

constructor calls only one time

* Constructor can have an access modifier.

- Does not have any return type, not even "void"
- Executed only once in the life span of an object.

In Java we do not write a constructor, Java adds a default constructors that initializes, all numeric member data by 0, character by space, boolean by false and reference types by null.

Garbage Collector → it operates periodically. if continious the program will be slow.

- Java does not have destructors, rather Java provides a utility called "**Garbage Collector**" that executes periodically and releases the unused memory.
- If you want our statements to be executed along with "**Garbage Collector**" you can provide them within: protected void finalize()
- "**Garbage Collector**" can also be invoked by programmer by calling: System.gc();

Getter/Setter VS public

```
class MyClass {
        private int a;
        public int b;
        public int getA() {
                return a;
        }
        public void setA(int a) {
                this.a = a > 100 ? 100 : a < 0 ? 0 : a;
        }
}
MyClass obj = new MyClass();
obj.setA(10000);
obj.b = 10000;
```

- With get/set we have options for read or write or both. But, public data always have both the options (read as well as write).
- With set it is possible to validate the data. But, public data cannot be validated.

this

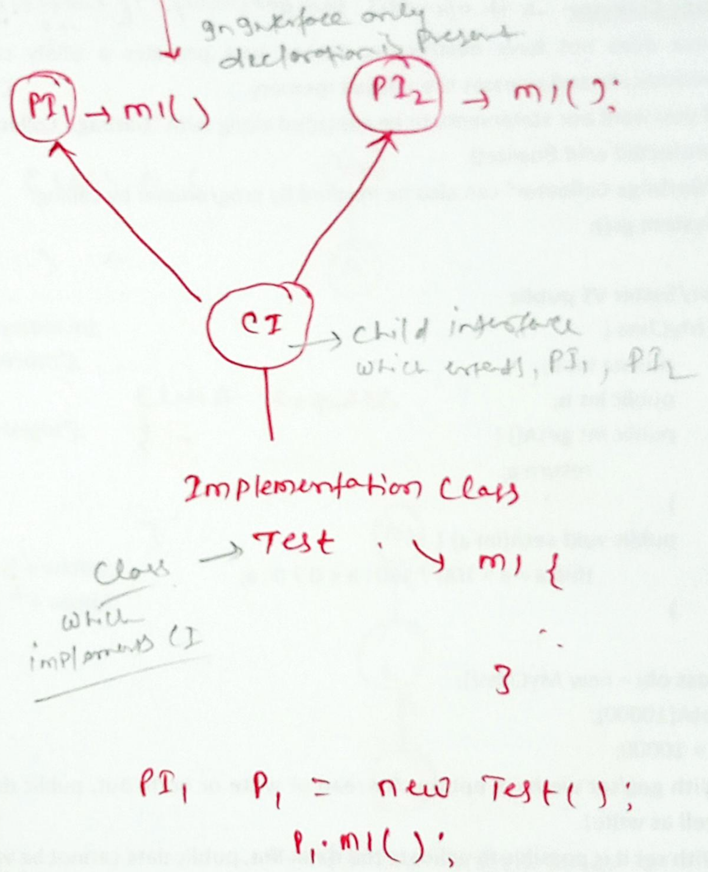
```
class MyClass {
        private int a;
        public int getA() {
                return a;
        }
        public void setA(int a) {
                this.a = a;
        }
}
MyClass obj1 = new MyClass();
obj1.setA(10);
```

```
//In the above line obj1 is the current context. Current context is the object in use.
MyClass obj2 = new MyClass();
obj2.setA(20);
//In the above line obj2 is the current context. Current context is the object in use.
```

In Java, when member data and argument shares a common name, member data is referred using "this". "this" refers to the current context. Current context is the object in use.

Example: Point

```
import java.util.Scanner;
class Point {
        protected int x;
        protected int y;
        public Point() {
        }
        public Point(int x, int y) {
                this.x = x;
                this.y = y;
        }
        public int getX () {
                return x;
        }
        public void setX (int x) {
                this.x = x;
        }
        public int getY () {
                return y;
        }
        public void setY (int y) {
                this.y = y;
        }
        public void acceptData() {
                Scanner a = new Scanner(System.in);
                System.out.println("Enter X  : ");
                x = a.nextInt();
                System.out.println("Enter Y  : ");
                y = a.nextInt();
        }
        public void showData() {
                System.out.println("X : " + x);
                System.out.println("Y : " + y);
        }
        public static void main(String args[]) {
                Point p = new Point();
                Point p1 = new Point(10,20);
```



In class multiple inheritance is not supported bcoz many class implements the same method.

but in interface only prototype is present. but implementation class only define one method.

members data / attributes
in Java it not support multiple inheritance (one child class can have only one parent class)


```
        p.showData();
        p1.showData();
    }
}
```

Inheritance

- Passing of properties from one class to another
- Properties include member data and member functions
- The class that gives the properties is called base/super/parent class
- The class that receives the properties is called derive/sub/child class
- Reasons : Reusability, Extending and Overriding

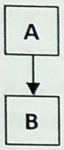
② Java does not support virtual data and virtual classes.

② Java does not support friend classes and friend function

③ Java does not support operator overloading, inline function and templates

Types of Inheritance

Single

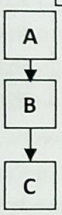


Multiple

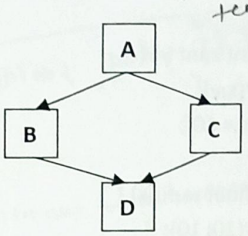
c.m1(); → ?

due to this ambiguity Java doesn't support multiple inheritance

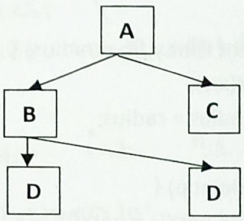
Multilevel



Hybrid



Hierarchical



* Diamond Allers Problem or Ambiguity problem

Function Overloading: Defining Function again with:

- Same name
- Different arguments (number / sequence / data type)
- Same class or base and derive class

method resolution done by JVM during runtime on the basis of runtime object.

Function Overriding: Defining Function again with

- Same name,
- Same arguments (number / sequence/data type)
- Different classes (base and derive)

In method overloading method resolution always takes cares by compiler based on reference types. not based on runtime object.

JVM is not responsible for method resolution.

that's why it is called compile time Polymorphism, static Polymorphism Early binding.

To inherit a class in Java "extends" keyword is used.

class <DeriveClass> extends <BaseClass>

From the view of derive class there are 3 categories of methods in the base.

1. Constructors: To call constructors of Base class "super" keyword is used, syntax is:

super(<arguments>);

Runtime Polymorphism
Dynamic Polymorphism
Late Binding.

Note: Only Derive class constructors can call constructors of base class (1st line)

2. Overridden Functions: To call overridden functions syntax is:

super.functionName(<args>);

3. Remaining/Normal Functions: To call functions apart from constructors and overridden functions, syntax is:

functionName(<args>);

Example: Circle

```
import java.util.Scanner;
class Circle extends Point {
        protected float radius;
        public Circle() {
                super(10, 10);
                radius = 10f;
        }
        public Circle(int x,int y) {
                super(x,y);
                radius = 10f;
        }
        public Circle(float radius) {
                super(10, 10);
                this.radius  = radius;
        }
        public Circle(int x,int y,float radius) {
                super(x,y);
                this.radius = radius;
        }
        public Circle(Point p) {
                super(p.getX(),p.getY());
                radius = 10f;
        }
        public Circle(Point p,float radius) {
                super(p.getX(),p.getY());
                this.radius = radius;
        }
        public float getRadius() {
                return radius;
        }
        public void setRadius(float radius) {
                this.radius = radius;
        }
        @Override
        public void acceptData() {
                super.acceptData();
                Scanner in = new Scanner(System.in);
                System.out.println("Enter Radius : ");
```

Cyclic Inheritance in Java not supported.

```
Class A extends A
{
}
```

```
Class A extends B
{
}
Class B extends A
{
}
```

```
        radius = in.nextFloat();
    }
    @Override
    public void showData() {
        super.showData();
        System.out.println("Radius : " + radius);
    }
    public float getArea() {
        return (float)(Math.PI * Math.pow(radius,2));
    }
    public float getCircumference() {
        return (float)(2 * Math.PI * radius);
    }
    public Point getCenter() {
        int x = getX();
        int y = getY();
        Point p = new Point(x,y);
        return p;
        // return new Point(getX(), getY());
    }
    public static void main(String args[]) {
        Circle c1 = new Circle(10,20,30f);
        Point p = new Point(1,2);
        Circle c2 = new Circle(p,3f);
        c1.showData();
        c2.showData();
        Point p2 = c2.getCenter();
        p2.showData();
        c2.getCenter().showData();
    }
}
```

```
class Test {
    public void m1(int i)
    {
    }
    public int m1(int j)
    {
        return 10;
    }
}
```

not allowed within the same class with same signature even return type is different.

static data

- Any data if declared as static is shared by all the objects.
- Only single copy of static data is created and this copy is owned and maintained by class.
- It is referred as class data rather than object data.
- To access static data, syntax is:

ClassName.dataName

However, it also be accessed using an object

- Benefits:
  - It saves memory.
  - It permits sharing.

class Abc {

```
    int a;
    int b;
    static int c;
}
Abc x = new Abc();
Abc y = new Abc();
Abc z = new Abc();
```

static blocks

- static blocks {} are used to initialize static data of a class.
- These blocks are executed as soon as the class is loaded in memory prior to main.

```
class TestStaticBlock {
    static int a;

    static {
        a = 10;
        System.out.println("A : " + a);
    }
    public static void main(String args[]) {
        a = 20;
        System.out.println("A : " + a);
    }
    static{
        a = 30;
        System.out.println("A : " + a);
    }
}
```

O/P - 10
30
20

static methods

- Any method that works only and only static data should be declared as static
- To invoke/call a static method, syntax is:

ClassName.methodName(<args>);

However, it also be invoked using an object

Note: NO OBJECT IS REQUIRED

- Functions that do not work on any member data, they are also declared as static (reusability)

Example: Create a class that counts the number of its objects created

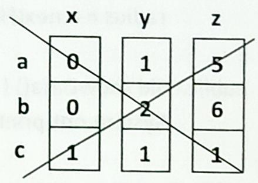
```
import java.util.Scanner;
class MyClassCtr {
    protected int a;
    protected int b;
    protected static int ctr;
    public MyClassCtr() {
        ctr++;
```

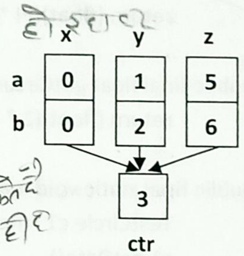
Normal function can access static data but static function can not use member data.

member function – member data को change करता है

```
}
public MyClassCtr(int a,int b) {
        this.a = a;
        this.b = b;
        ctr++;
}
public static int getCtr() {
        return ctr;
}
public void getData() {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter A : ");
        a = in.nextInt();
        System.out.println("Enter B : ");
        b = in.nextInt();
}
public void showData() {
        System.out.println("A is : " + a);
        System.out.println("B is : " + b);
}
public static void main(String args[]) {
    1   MyClassCtr x = new MyClassCtr();   → पहला constructor call
    2   MyClassCtr y = new MyClassCtr(1,2);  → दूसरा
    3   MyClassCtr z = new MyClassCtr(5,6);  — तिसरा
        System.out.println("Counter is : " + MyClassCtr.getCtr());
}
}
```

x,y,z से भि call कर सकते है क्योंकि memory एक है





final data

- Any data if declared as final cannot be changed.
- It is used to declare constants in Java.
- final data if declared within class should be declared as static.

At compile time final value will be replaced by the value. At runtime final variable not in variable form.

final methods

- Any method if declared as final cannot be overridden.

final class

- Any class if declared as final cannot be inherited.
- It is used to end the hierarchy.

derive class से main नहीं लिखा पाओगे

Example: TestCircle

```
import java.util.Scanner;
class TestCircle {
        protected float radius;
        public static final float PI = 3.14f;
        public TestCircle() {
```

नया कुछ नहीं होगा परंतु

sequence change कर सकते है

एक बार ही लिखा static

static एक copy बनाते और final कोई change न करे

```
    }
    public TestCircle(float radius) {
        this.radius = radius;
    }
    public float getRadius() {
        return radius;
    }
    public void setRadius(float radius) {
        this.radius = radius;
    }
    public void getData() {
        Scanner a = new Scanner(System.in);
        System.out.println("Enter Radius :");
        radius = a.nextFloat();
    }
    public void showData() {
        System.out.println("radius is : " + radius);
    }
    public final float getArea() {
        return (float)(PI * Math.pow(radius,2));
    }
    public final float getCircumference() {
        return (float)(2 * PI * radius);
    }
    public final static void main(String args[]) {
        TestCircle c1 = new TestCircle();
        c1.getData();
        c1.showData();
        System.out.println("Area is :  " + c1.getArea());
    }
}
```

abstract methods

- If any method is declared as abstract, it should be compulsorily overridden.
- These methods does not contain any statements, they only enforces symmetry in hierarchy.

abstract classes

- If any class contains at least one abstract method, it should be compulsorily declared as abstract.
- class created without abstract methods can also be declared as abstract.
- No object of abstract class can be created; it can only store reference to derive class objects.
- abstract class can have data constructors and methods that are used by derive classes.
- These classes are used to start the hierarchy and define rules for the hierarchy in form of abstract methods.

Example: AbstractA.java

```
abstract class AbstractA {
        public abstract void acceptData();
        public abstract void showData();
}
```

Example: DerivedA.java

```
import java.util.Scanner;
class DerivedA extends AbstractA {
        protected int a;
        public DerivedA(){}
        public DerivedA(int a) {
                this.a = a;
        }
        public void acceptData() {
                Scanner in = new Scanner(System.in);
                System.out.println("Enter A :");
                a = in.nextInt();
        }
        public void showData()  {
                System.out.println("A is : " + a);
        }
        public static void main(String args[]) {
                AbstractA a = new DerivedA(10);
                DerivedA b = new DerivedA(20);
// WRONG        AbstractA c = new AbstractA();
                a.showData();
                b.showData();
        }
}
```

Assignment

Theory Questions

1. Describe OOPS.
2. What are classes? Why do we write a constructor in a class? Does Java provide destructors?
3. What are access specifiers? Explain the types of access specifiers available in Java.
4. What is heritance and what are its types? Explain the benefits of inheritance.
5. Explain differences between function overloading and function overriding.
6. Explain static data, static blocks and static functions.
7. What are abstract functions and abstract classes?
8. Explain why we need final data, final functions and final classes.

Practical Questions